

APPLICATION GUIDE

Horizon Bank Cache App

React · Express · PostgreSQL · Redis · Docker
Architecture, layers and flows - with exercises

ICICI Bank Learning Academy

Enterprise Data Engineering with SQL, Redis & Kafka

Module 07 · NoSQL & Redis · Instructor-led · For classroom use

How to use this guide

Horizon Bank is a small but complete net-banking dashboard built to show Redis caching in a real application: a **React** front end, an **Express** back end, **PostgreSQL** as the system of record and **Redis** as the cache - all four running in **Docker**. It has one screen with two menu items: *Transactions* (the account statement - always served from the cache when it can be) and *Fund Transfer* (moves money in PostgreSQL, then brings the cache up to date).

This guide explains how it is built: the architecture, each layer, both request flows step by step, the cache design, the Docker setup, and how to test it from the browser and the command line. The companion `Redis_Caching_Guide.pdf` teaches the Redis commands this app issues.

Section	Title
1	What the application does
2	Architecture
3	Layers and folder structure
4	Data model and seed data
5	Cache design and configuration
6	Flow 1 - transaction enquiry (cache-aside)
7	Flow 2 - fund transfer (database first, cache second)
8	REST API reference
9	Docker setup
10	Run it and test it - browser and command line
11	Failure drills - Redis down, cache off, invalidate mode
12	Troubleshooting
13	Exercises with solutions

PREREQUISITES Docker Desktop running, ports 8080, 4000, 5433 and 6380 free, and a bash shell (Git Bash or WSL) for the scripts. No Node.js or PostgreSQL install is needed on the host - everything builds inside containers.

```
$ cd Redis_Training_Pack/cache_banking_app
$ docker compose up -d --build
# open http://localhost:8080
```

NOTE Horizon Bank is a fictional bank. Every customer name, account number and amount is synthetic training data.

SECTION 1

What the application does

One dashboard for one logged-in customer (Arjun Mehta, customer 1, three accounts). The layout follows a typical net-banking screen: a branded top bar with the customer and last login, a left menu, account cards with balances, and the working area.

Horizon Bank
PERSONAL NET BANKING

Secure session | Welcome, Arjun Mehta | Last login: 24 Sept 2026, 04:15 am | Logout

ACCOUNTS

- Transactions (Account statement)
- Fund Transfer (Own & other accounts)

Home > Accounts > Account Statement

My Accounts
Total available balance ₹13,85,339.87

Redis cache HIT · 3.9 ms · TTL 304s | Refresh

ACCOUNT TYPE	ACCOUNT NO.	AVAILABLE BALANCE	BRANCH	ACCOUNT ID
SAVINGS ACCOUNT	XXXX XXXX 0101	₹1,18,053.04	Andheri West, Mumbai	HRZN0000123
SALARY ACCOUNT	XXXX XXXX 0102	₹8,34,973.16	Bandra Kurla, Mumbai	HRZN0000145
CURRENT ACCOUNT	XXXX XXXX 0103	₹4,32,313.67	Fort, Mumbai	HRZN0000101

Account Statement
Savings Account - XXXX XXXX 0101 - Andheri West, Mumbai

Cache MISS - loaded from PostgreSQL · 10.3 ms · TTL 300s | Enquire again

All Credits Debits | Search description or reference no.

Cr ₹1,00,397.14 Dr ₹1,09,759.58

DATE	DESCRIPTION	REF. NO.	WITHDRAWAL (DR)	DEPOSIT (CR)	BALANCE
25 Sept 2026 06:15 am	TRF From 602501000102 - Monthly savings	TRF297139186441		₹25,000.00	₹1,18,053.04
25 Sept 2026 06:15 am	TRF To 602501000201 - Rent - October	TRF297139076331	₹18,000.00		₹93,053.04
24 Sept 2026 09:14 pm	UPI/BookMyShow/Movies	UPI260924778726	₹203.78		₹1,11,053.04
23 Sept 2026 12:56 pm	UPI/Uber India/Ride	UPI260923738439	₹3,890.27		₹1,11,256.82
21 Sept 2026 10:54 pm	UPI/BookMyShow/Movies	UPI260921949786	₹1,165.35		₹1,15,147.09
21 Sept 2026 12:34 am	UPI/IRCTC/Train tickets	UPI260920715297	₹2,684.93		₹1,16,312.44
19 Sept 2026 03:27 pm	UPI/BigBasket/Groceries	UPI260919192926	₹4,943.91		₹1,18,997.37
18 Sept 2026 12:40 am	UPI/Swiggy/Food order	UPI260917644896	₹2,901.64		₹1,23,941.28
17 Sept 2026 01:22 am	UPI/BookMyShow/Movies	UPI260916219453	₹2,302.20		₹1,26,842.92

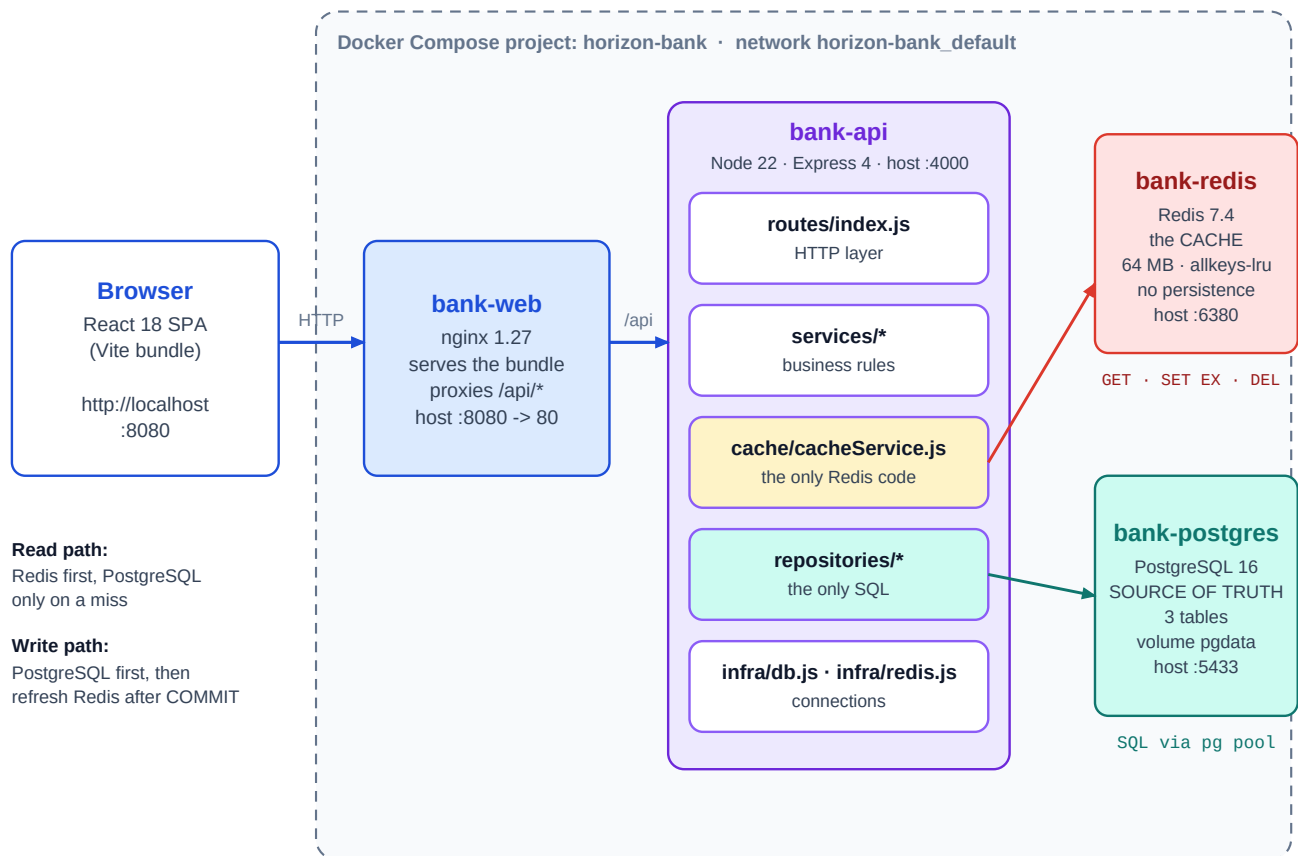
Screen element	API call	Cached as
My Accounts - three cards, total balance	GET /api/customers/1/accounts	bank:accounts:1
Transactions - latest 50, filter Credits/Debits, search	GET /api/accounts/:no/transactions	bank:txns:<accountNo>
Fund Transfer - details -> review -> acknowledgement	POST /api/transfers	(write - updates the keys)
Green / amber badge	X-Cache header + source field	HIT = Redis, MISS = PostgreSQL
Cache Inspector (training strip)	GET /api/cache/stats, /api/cache/keys	shows live bank:* keys + TTL

The badge on every panel is the teaching device: the first enquiry of an account shows an amber **Cache MISS - loaded from PostgreSQL**; **Enquire again** turns it green - **Redis cache HIT** - with the key's remaining TTL.

SECTION 2

Architecture

Four containers on one Docker network. The browser only ever talks to nginx; nginx serves the React bundle and forwards `/api/*` to Express, so the UI and API share one origin. Inside Express, exactly one module talks to Redis and exactly one layer talks to PostgreSQL.



Container	Image	Role	Host port
bank-web	nginx:1.27-alpine (multi-stage build)	Serves the React bundle; reverse-proxies /api	8080
bank-api	node:22-alpine	Express REST API; cache-aside logic	4000
bank-postgres	postgres:16-alpine	System of record: customers, accounts, transactions	5433
bank-redis	redis:7-alpine	Cache only - 64 MB, allkeys-lru, persistence off	6380

PORTS Host ports 5433 and 6380 are deliberate: they avoid a PostgreSQL installed on the laptop (5432) and the Redis containers from the other Module 07 labs (6379). Inside the Docker network the services still use the standard ports - the API connects to `postgres:5432` and `redis:6379` by container name.

SECTION 3

Layers and folder structure

Each layer has one job and only calls the layer below it. The rule that matters for caching: **services decide what to cache and when to refresh it; cacheService decides how**. Repositories never know a cache exists, so the SQL can be tested and reasoned about on its own.

PRESENTATION	web/src/components/*.jsx Header, Sidebar, AccountCards, TransactionsView, TransferView, CacheInspector - renders state, never talks to Redis or SQL
CLIENT API	web/src/api.js fetch('/api/...') wrappers + INR / date formatting
EDGE	web/nginx.conf Static files; /api/* reverse-proxied to bank-api:4000 (same origin, no CORS)
ROUTES	api/src/routes/index.js Parse request -> call a service -> JSON; adds X-Cache and X-Response-Time headers
SERVICES	api/src/services/*.js accountService: what to cache and under which key; transferService: DB transaction, then cache update
CACHE	api/src/cache/cacheService.js Cache-aside getOrLoad(), refresh(), invalidate(), stats(), listKeys() - the only file that imports the Redis client
REPOSITORIES	api/src/repositories/*.js Parameterised SQL only; knows nothing about Redis
DATA	PostgreSQL 16 + Redis 7 customers / accounts / transactions · bank:* keys with TTL

```

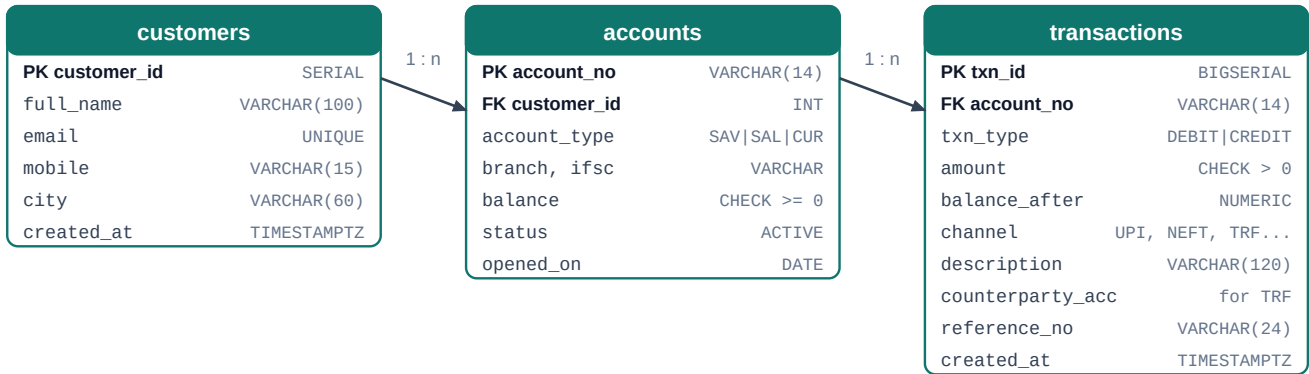
cache_banking_app/
|-- docker-compose.yml      4 services, healthchecks, cache env vars
|-- db/init/
|   |-- 01_schema.sql      3 tables + 2 indexes
|   |-- 02_seed.sql        6 customers, 8 accounts, 507 transactions
|-- redis/redis-cache.conf  maxmemory, allkeys-lru, no persistence
|-- api/                    Express back end
|   |-- Dockerfile
|   |-- src/
|       |-- server.js      wires layers, starts HTTP, connects Redis
|       |-- config/index.js every tunable from env vars
|       |-- infra/db.js     pg Pool + withTransaction()
|       |-- infra/redis.js  one shared node-redis client
|       |-- cache/cacheService.js  getOrLoad / refresh / invalidate / stats
|       |-- repositories/   accountRepository.js, transactionRepository.js
|       |-- services/       accountService.js, transferService.js
|       |-- routes/index.js 9 endpoints
|       |-- middleware/errors.js  HttpError, asyncRoute, errorHandler, requestLog
|-- web/                    React front end
|   |-- Dockerfile, nginx.conf, vite.config.js, index.html
|   |-- src/  main.jsx, App.jsx, api.js, styles.css, components/ (7 files)
|-- scripts/cache-demo.sh   command-line walk-through

```

SECTION 4

Data model and seed data

Three tables - deliberately few. Money is NUMERIC(14,2), never floating point; CHECK (balance >= 0) means the database itself refuses an overdraft even if the application had a bug.



Index: idx_txn_account_time (account_no, created_at DESC, txn_id DESC) - serves the statement query exactly.

Index: idx_accounts_customer (customer_id) - serves the account list.

Account	Holder	Type	Branch	Seeded rows
602501000101	Arjun Mehta (logged in)	SAVINGS	Andheri West, Mumbai	95
602501000102	Arjun Mehta	SALARY	Bandra Kurla, Mumbai	60 (incl. monthly salary)
602501000103	Arjun Mehta	CURRENT	Fort, Mumbai	56
602501000201	Priya Sharma	SAVINGS	Connaught Place, Delhi	60
602501000301	Rahul Verma	SAVINGS	Koregaon Park, Pune	60
602501000401	Sneha Iyer	SAVINGS	T Nagar, Chennai	60
602501000501	Vikram Rao	CURRENT	MG Road, Bengaluru	56
602501000601	Ananya Gupta	SAVINGS	Park Street, Kolkata	60

02_seed.sql generates 120 days of UPI, POS, ATM, NEFT, IMPS, NACH, salary and interest entries with setseed(0.42), so every fresh container gets the same amounts and descriptions (the dates move with the day you start it). It then computes balance_after for every row with a window function, and sets each account's balance to the sum - so the statement's running balance always reconciles.

```

-- running balance after each transaction (from 02_seed.sql)
UPDATE transactions t SET balance_after = r.running
FROM (SELECT txn_id, SUM(CASE WHEN txn_type = 'CREDIT' THEN amount ELSE -amount END)
      OVER (PARTITION BY account_no ORDER BY created_at, txn_id) AS running
      FROM transactions) r
WHERE r.txn_id = t.txn_id;

```

SECTION 5

Cache design and configuration

Keys

Key	Value	TTL	Written on	Refreshed / deleted when
bank:txns:<accountNo>	JSON array - latest 50 transactions	300 + 0..30 s	Statement MISS	A transfer debits or credits that account
bank:accounts:<customer Id>	JSON array - accounts with balances	300 + 0..30 s	Account-list MISS	A transfer touches any account of that customer
bank:stats:hits / :misses	Integer (INCR)	none	Every cached read	Clear cache button

Keys are *computable*: the transfer service knows both account numbers and both owners, so it can name every key it affects without a SCAN. The value is one JSON string because a statement is always read whole.

Configuration parameters

Where	Parameter	Value	Effect
docker-compose.yml	CACHE_ENABLED	true	false = bypass Redis entirely (source: database (cache off))
	CACHE_TTL_SECONDS	300	Base lifetime of every cached value
	CACHE_TTL_JITTER_SECONDS	30	Random extra 0..30 s - stops keys expiring in the same second
	CACHE_ON_WRITE	refresh	refresh = SET new values after COMMIT; invalidate = DEL them
	STATEMENT_SIZE	50	Rows per cached statement
redis-cache.conf	CACHE_KEY_PREFIX	bank	Namespace for every key
	maxmemory	64mb	Memory ceiling
	maxmemory-policy	allkeys-lru	Evict least-recently-used keys when full
	save / appendonly	"" / no	No persistence - a restart empties the cache, which is fine
	lazyfree-lazy-*	yes	Free evicted/expired/deleted values off the main thread

Every compose value can be overridden from the shell without editing a file:

```
$ CACHE_TTL_SECONDS=30 docker compose up -d api      # short TTL, watch keys expire
$ CACHE_ON_WRITE=invalidate docker compose up -d api  # delete instead of refresh
$ CACHE_ENABLED=false docker compose up -d api       # no cache at all
$ docker compose up -d api                            # back to the defaults
```

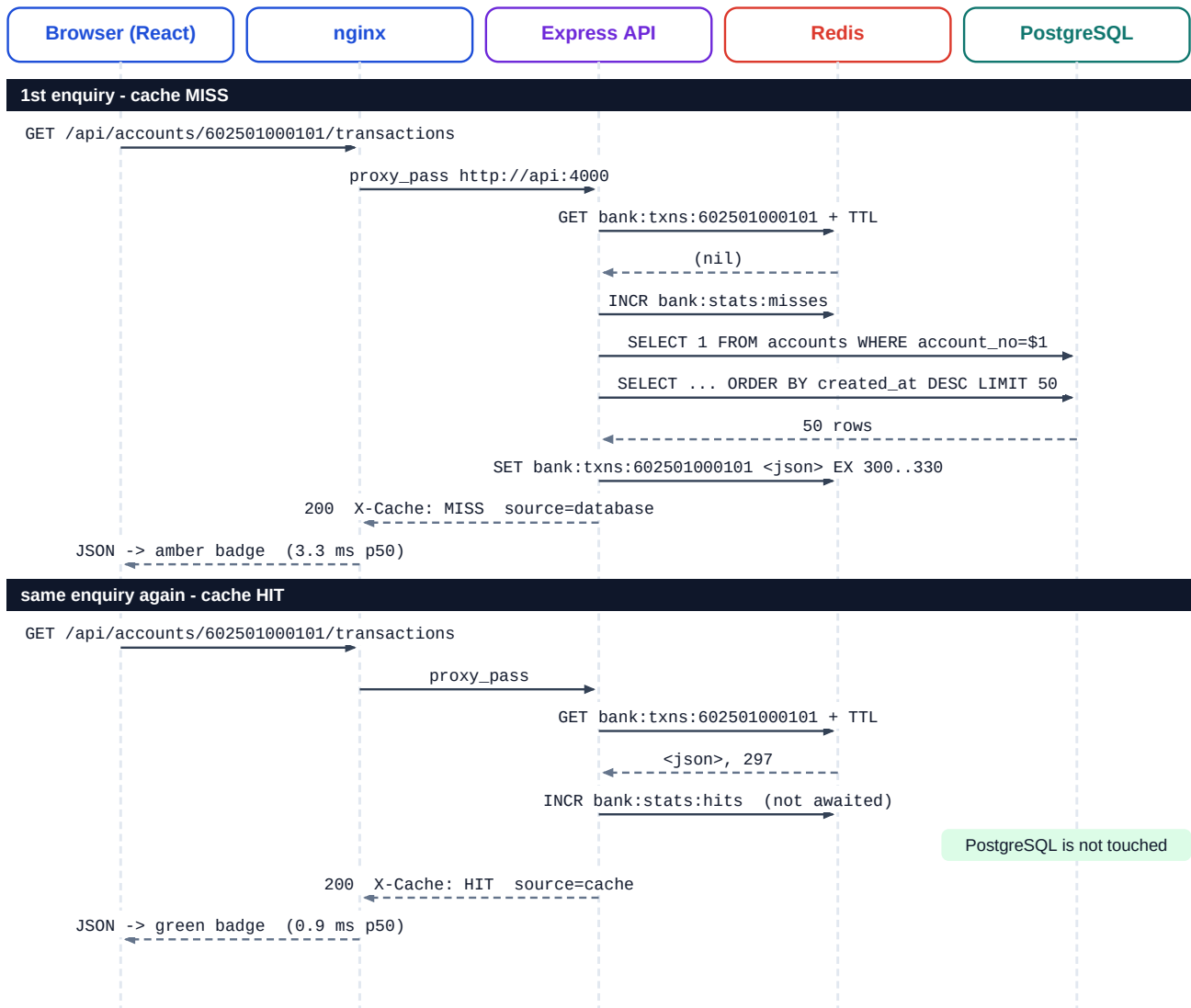
What happens when things fail

Situation	Behaviour	Response source
Redis down at a read	Served from PostgreSQL; API keeps running and reconnects in the background	database (redis down)
Redis write fails after a MISS	Response still returned; error logged	database
Cache update fails after a transfer	Transfer still succeeds (money moved); keys deleted if possible; TTL bounds any staleness	-
Account does not exist	404 from the loader - nothing is cached	-
PostgreSQL down	500 on a MISS; cached keys keep being served until their TTL	cache

SECTION 6

Flow 1 - transaction enquiry (cache-aside)

Measured with 100 requests each on this machine: MISS median **3.3 ms** (p95 5.4 ms), HIT median **0.9 ms** (p95 1.6 ms), timed inside the API.



The code - cacheService.getOrLoad()

```

// api/src/cache/cacheService.js
export async function getOrLoad(key, loader) {
  if (!config.cache.enabled) {
    return { data: await loader(), source: 'database (cache off)', ttl: null, key };
  }
  if (!redis.isReady) {
    return { data: await loader(), source: 'database (redis down)', ttl: null, key };
  }

  let cached;
  let ttl;
  try {
    // GET and TTL issued in the same tick -> node-redis pipelines them into one round trip
  
```

```

    [cached, ttl] = await Promise.all([get(key), redis.ttl(key)]);
  } catch (err) {
    // Redis failed mid-request: the database still answers
    console.error('[cache] read failed, falling back to database:', err.message);
    return { data: await loader(), source: 'database (redis down)', ttl: null, key };
  }

  if (cached !== null) {
    countHit(true).catch(() => {}); // metrics must never slow down or fail a read
    console.log(`[cache] HIT   ${key} (ttl ${ttl}s)`);
    return { data: cached, source: 'cache', ttl, key };
  }

  countHit(false).catch(() => {});
  const data = await loader(); // a 404 from the loader propagates as-is
  try {
    ttl = await set(key, data);
    console.log(`[cache] MISS ${key} -> loaded from PostgreSQL, SET EX ${ttl}`);
  } catch (err) {
    ttl = null;
    console.error('[cache] write failed (response still served):', err.message);
  }
  return { data, source: 'database', ttl, key };
}

```

The caller - `accountService.getStatement()`

```

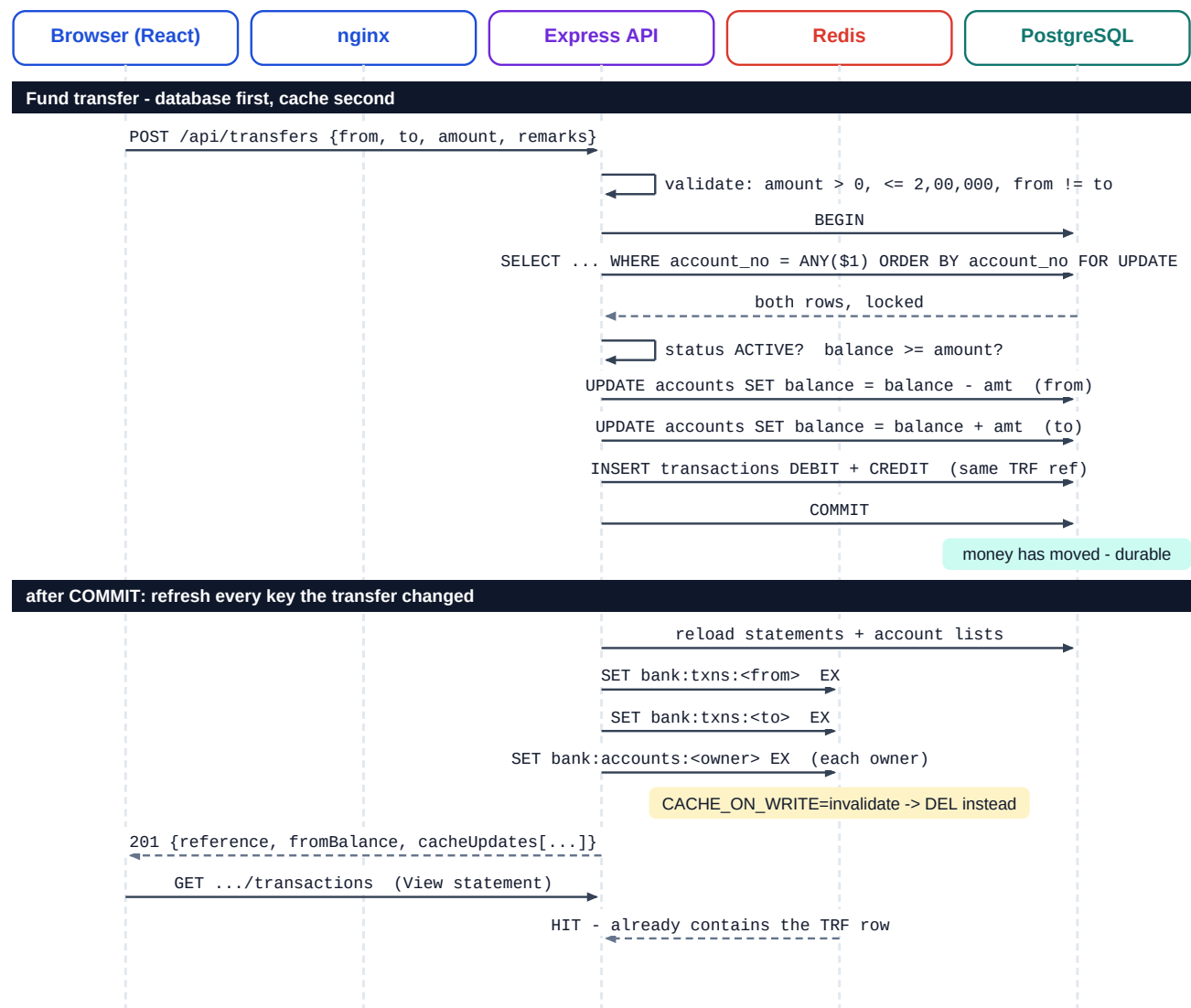
// api/src/services/accountService.js
/** Recent transactions - cached as bank:txns:<accountNo>. */
export async function getStatement(accountNo) {
  return cache.getOrLoad(cache.keys.statement(accountNo), async () => {
    if (!(await accounts.findAccountHolder(accountNo))) {
      throw new HttpError(404, `Account ${accountNo} not found`);
    }
    return loadStatement(accountNo);
  });
}

```

LESSON The existence check lives *inside* the loader, so it runs only on a MISS. An earlier draft checked the account in PostgreSQL before looking in Redis - every HIT still paid for a database query. Moving the check cut the HIT median from 2.8 ms to 0.9 ms. A cache that still queries the database on a hit is not a cache.

SECTION 7

Flow 2 - fund transfer (database first, cache second)



Four rules the transfer service follows

1. **The balance check reads PostgreSQL, never Redis** - `SELECT ... FOR UPDATE` locks both rows for the rest of the transaction.
2. **Lock in a fixed order** (`ORDER BY account_no`) - two opposite transfers `A->B` and `B->A` cannot deadlock each other.
3. **Touch the cache only after COMMIT** - updating Redis first could publish a transfer that then rolls back.
4. **A cache failure never fails the transfer** - the money has moved; the TTL bounds how long a missed update can be visible.

```

// api/src/services/transferService.js
async function updateCache({ fromAccount, toAccount, owners }) {
  try {
    if (config.cache.onWrite === 'invalidate') {
      return await cache.invalidate(
        cache.keys.statement(fromAccount),
        cache.keys.statement(toAccount),

```

```

    ...owners.map((c) => cache.keys.accounts(c)),
  );
}
const jobs = [
  cache.refresh(cache.keys.statement(fromAccount), () => loadStatement(fromAccount)),
  cache.refresh(cache.keys.statement(toAccount), () => loadStatement(toAccount)),
  ...owners.map((c) => cache.refresh(cache.keys.accounts(c), () => loadAccounts(c))),
];
return (await Promise.all(jobs)).filter(Boolean);
} catch (err) {
  // The money has moved; a failed cache update must not fail the transfer.
  // Try to delete the keys so no stale balance is served; TTL cleans up the rest.
  console.error('[cache] update after transfer failed:', err.message);
  return cache.invalidate(
    cache.keys.statement(fromAccount),
    cache.keys.statement(toAccount),
    ...owners.map((c) => cache.keys.accounts(c)),
  ).catch(() => []);
}
}
}

```

CACHE_ON_WRITE	After a transfer	Next statement read	Choose when
refresh (default)	SET the 3-4 affected keys with fresh data	HIT - already shows the transfer	Reads right after a write are likely (the user opens the statement)
invalidate	DEL the affected keys	MISS - reloads from PostgreSQL	Writes are frequent and many keys may never be read again

SECTION 8

REST API reference

Method	Path	Cached	Returns
GET	/api/health	-	{api, db, redis} up/down
GET	/api/customers/:id	-	Profile: name, email, mobile, city
GET	/api/customers/:id/accounts	yes	{source, key, ttl, tookMs, data: [accounts]}
GET	/api/accounts/:no/transactions	yes	{source, key, ttl, tookMs, data: [50 txns]}
GET	/api/accounts/:no/lookup	-	Beneficiary check: masked holder name, branch, IFSC
POST	/api/transfers	writes	201 {reference, amount, fromBalance, createdAt, cacheUpdates[]}
GET	/api/cache/stats	-	hits, misses, hitRate, usedMemory, maxMemory, policy, onWrite
GET	/api/cache/keys	-	[[{key, ttl, bytes}] for every bank:* key
DELETE	/api/cache	-	{deleted: n} - removes every bank:* key

Cached reads also set two headers: X-Cache: HIT|MISS and X-Response-Time.

```
$ curl -si localhost:4000/api/accounts/602501000101/transactions | head -6
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
X-Cache: MISS
X-Response-Time: 15.5ms
Content-Type: application/json; charset=utf-8
```

Transfer request and validation errors

```
$ curl -s -X POST localhost:4000/api/transfers -H 'Content-Type: application/json' \
-d '{"fromAccount":"602501000101","toAccount":"602501000103","amount":99999999}'
{"error":"Maximum per transfer is Rs 2,00,000"}
... "amount":150000 ...
{"error":"Insufficient balance. Available: Rs 1,09,403.04"}
$ curl -s localhost:4000/api/accounts/999/transactions
{"error":"Account 999 not found"}
```

Status	When
400	Missing accounts, same account both sides, amount <= 0, > 2 decimals, > Rs 2,00,000
404	Unknown account or customer
409	An account is not ACTIVE
422	Insufficient balance

SECTION 9

Docker setup

```
// docker-compose.yml
api:
  build: ./api
  container_name: bank-api
  environment:
    PORT: 4000
    PGHOST: postgres
    PGPORT: 5432
    PGDATABASE: horizonbank
    PGUSER: bank
    PGPASSWORD: bank_pass
    REDIS_URL: redis://redis:6379
    # ---- cache configuration (see Horizon_Bank_App_Guide.pdf, section 5) ----
    # Override any of these from the shell, e.g.
    #   CACHE_ON_WRITE=invalidate docker compose up -d api
    CACHE_ENABLED: ${CACHE_ENABLED:-true}
    CACHE_TTL_SECONDS: ${CACHE_TTL_SECONDS:-300}
    CACHE_TTL_JITTER_SECONDS: ${CACHE_TTL_JITTER_SECONDS:-30}
    CACHE_KEY_PREFIX: bank
    CACHE_ON_WRITE: ${CACHE_ON_WRITE:-refresh}
    STATEMENT_SIZE: ${STATEMENT_SIZE:-50}
  ports:
    - "4000:4000"
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  restart: unless-stopped
```

Feature	Why it is there
healthcheck on postgres (pg_isready) and redis (PING)	The API must not start before its dependencies accept connections
depends_on: condition: service_healthy	Turns those healthchecks into start-up ordering
./db/init mounted at /docker-entrypoint-initdb.d	Schema + seed run automatically - only the first time the volume is empty
named volume pgdata	Data survives docker compose down; removed by down -v
redis-cache.conf mounted read-only	Cache settings are versioned with the code, not typed by hand
web: multi-stage Dockerfile	Node builds the bundle; only nginx + static files ship
\${VAR:-default} in environment	Change cache settings from the shell for experiments

Task	Command
Build and start everything	<code>docker compose up -d --build</code>
Status and health	<code>docker compose ps</code>
Follow the API's [cache] log lines	<code>docker logs -f bank-api</code>
redis-cli into the cache	<code>docker exec -it bank-redis redis-cli</code>
psql into the database	<code>docker exec -it bank-postgres psql -U bank -d horizonbank</code>
Rebuild only the API after a code change	<code>docker compose up -d --build api</code>
Stop, keep data	<code>docker compose down</code>
Stop and re-seed from scratch	<code>docker compose down -v then up -d --build</code>

SECTION 10

Run it and test it

Step 1 - start and check health

```
$ docker compose up -d --build
$ docker compose ps --format 'table {{.Name}}\t{{.Status}}'
NAME                STATUS
bank-api            Up 8 minutes
bank-postgres       Up 18 minutes (healthy)
bank-redis          Up 10 minutes (healthy)
bank-web            Up 7 minutes
$ curl -s localhost:8080/api/health
{"api":"up","db":"up","redis":"up"}
```

Step 2 - in the browser (http://localhost:8080)

1. The account cards load - amber **Cache MISS** badge the very first time. Click **Refresh**: green **Redis cache HIT**.
2. **Transactions** shows the savings account. Click **Enquire again** - MISS turns into HIT and the time drops. Click another account card: that account's first enquiry is a MISS.
3. Open the **Cache Inspector** strip at the bottom: one bank : txns : * key per account you viewed, each with a TTL counting down from ~300.
4. **Fund Transfer**: choose *Other Horizon Bank account*, enter 602501000201, **Verify** (shows "Priya S."), amount 2500, remarks, **Continue**, **Confirm & Transfer**.
5. The acknowledgement lists the four keys refreshed after COMMIT. Click **View statement**: the statement is a **HIT** and its first row is the new TRF debit - the cache was updated, not just emptied.
6. Click **Clear cache** in the inspector and enquire again - back to a MISS.

The screenshot displays the Horizon Bank Personal Net Banking interface. At the top, the header includes the bank logo, "PERSONAL NET BANKING", a "Secure session" indicator, a welcome message for "Arjun Mehta" with the last login time, and a "Logout" button. The left sidebar contains navigation links for "ACCOUNTS", "Transactions", "Account statement", "PAYMENTS", and "Fund Transfer". The main content area shows "My Accounts" with a total available balance of ₹13,85,339.87. Three account cards are visible: "SAVINGS ACCOUNT" (XXXX XXXX 0101) with an available balance of ₹1,18,053.04, "SALARY ACCOUNT" (XXXX XXXX 0102) with ₹8,34,973.16, and "CURRENT ACCOUNT" (XXXX XXXX 0103) with ₹4,32,313.67. Below the accounts, the "Fund Transfer" section is active, showing a three-step process: "1 Transfer details", "2 Review & confirm", and "3 Acknowledgement". The "Transfer details" step is currently selected, showing a "Transfer from" dropdown set to "Savings Account - XXXX XXXX 0101 - ₹1,18,053.04", a "Transfer to" dropdown set to "My own account", a "Credit account" dropdown set to "Salary Account - XXXX XXXX 0102", and an "Amount (₹)" input field set to "0.00". There is also a "Remarks (optional)" field with the placeholder text "e.g. Rent for October". A "Continue" button is at the bottom right of the form. At the very bottom, a "Cache Inspector" strip is visible, indicating it is a training view.

Step 3 - from the command line

```
$ bash scripts/cache-demo.sh
== 1. Start clean: delete every bank:* key
{"deleted":6}
== 2. First enquiry -> MISS, loaded from PostgreSQL, stored in Redis
source=database ttl=300s took=22.2ms
== 3. Same enquiry again -> HIT, served from Redis
source=cache ttl=300s took=2.4ms
== 5. Transfer Rs 100 to 602501000201 -> DB COMMIT, then cache refreshed
{"reference":"TRF295949631951",...,"cacheUpdates":[{"key":"bank:txns:602501000101",
  "action":"refreshed","ttl":305}, ... 4 keys ...]}
== 6. Enquire again -> still a HIT, and the newest row is the transfer
"source":"cache"
"description":"TRF/To 602501000201 - cli demo"
== 7. Hit / miss counters
{"redis":"up",...,"hits":2,"misses":1,"hitRate":66.7,"usedMemory":"1.26M",
  "maxMemory":"64.00M","policy":"allkeys-lru"}
```

(The first MISS of a fresh container is slower - 15 to 22 ms - because PostgreSQL's cache and the pg pool are cold; steady-state misses are ~3 ms.)

Step 4 - the API's own log

```
$ docker logs bank-api
[cache] MISS bank:txns:602501000101 -> loaded from PostgreSQL, SET EX 329
[api] GET /api/accounts/602501000101/transactions 200 21.8ms
[cache] HIT bank:txns:602501000101 (ttl 329s)
[api] GET /api/accounts/602501000101/transactions 200 3.5ms
[cache] UPDATE bank:txns:602501000102 -> refreshed after write, SET EX 326
[cache] UPDATE bank:accounts:1 -> refreshed after write, SET EX 320
[cache] UPDATE bank:txns:602501000101 -> refreshed after write, SET EX 306
[api] POST /api/transfers 201 24.2ms
```

Both accounts in that transfer belong to customer 1, so only one bank:accounts:* key needed a refresh - the service de-duplicates owners.

SECTION 11

Failure drills

Redis goes down

```
$ docker stop bank-redis
$ curl -s localhost:4000/api/accounts/602501000101/transactions | cut -c1-80
{"source":"database (redis down)","key":"bank:txns:602501000101","ttl":null}
$ curl -s localhost:4000/api/health
{"api":"up","db":"up","redis":"down"}
$ docker start bank-redis
$ docker logs bank-api 2>&1 | grep redis | tail -2
[redis] error: connect ECONNREFUSED 172.20.0.2:6379
[redis] connected redis://redis:6379
```

The dashboard keeps working (slower, amber badges) and recovers by itself. Because Redis runs without persistence, it restarts empty - the first read of every key is a MISS again.

Cache switched off

```
$ CACHE_ENABLED=false docker compose up -d api
$ curl -s localhost:4000/api/accounts/602501000101/transactions | cut -c1-60
{"source":"database (cache off)","key":"bank:txns:602501000101","ttl":null}
```

Invalidate instead of refresh

```
$ CACHE_ON_WRITE=invalidate docker compose up -d api
$ curl -s -X POST localhost:4000/api/transfers -H 'Content-Type: application/json' \
  -d '{"fromAccount":"602501000101","toAccount":"602501000103","amount":50}' | grep -o
'"cacheUpdates.*'
  "cacheUpdates":[{"key":"bank:txns:602501000101","action":"deleted","ttl":null},
    {"key":"bank:txns:602501000103","action":"deleted","ttl":null},
    {"key":"bank:accounts:1","action":"deleted","ttl":null}]
$ curl -s localhost:4000/api/accounts/602501000101/transactions | cut -c1-60
{"source":"database","key":"bank:txns:602501000101","ttl":30}
$ docker compose up -d api # restore the defaults
```

SECTION 12

Troubleshooting

Symptom	Cause	Fix
Bind for 0.0.0.0:5433 / 6380 / 8080 failed: port is already allocated	Another container or service uses the port	Stop it, or change the left-hand port in docker-compose.yml
Seed changes have no effect	initdb scripts only run on an empty volume	docker compose down -v, then up -d --build
Badge always amber, source "database (redis down)"	bank-redis not running	docker compose ps; docker compose up -d redis
Badge always amber, source "database (cache off)"	CACHE_ENABLED=false still set in your shell	unset CACHE_ENABLED; docker compose up -d api
UI shows old code after an edit	The web image was not rebuilt	docker compose up -d --build web
Redis refuses to start: wrong number of arguments	A comment on the same line as a directive in redis-cache.conf	Move the comment to its own line
Transfer fails with 422	Insufficient balance - the check is in PostgreSQL	Transfer less, or from the salary account

SECTION 13

Exercises with solutions

EXERCISE 1

Watch a key expire

TASK Run the API with a 20-second TTL and no jitter. Enquire once, watch the TTL count down in redis-cli, and show that the enquiry after it expires is a MISS again.

Solution

```
$ CACHE_TTL_SECONDS=20 CACHE_TTL_JITTER_SECONDS=0 docker compose up -d api
$ curl -s localhost:4000/api/accounts/602501000102/transactions >/dev/null
$ docker exec bank-redis redis-cli TTL bank:txns:602501000102
20
$ sleep 21; docker exec bank-redis redis-cli TTL bank:txns:602501000102
-2
$ curl -s localhost:4000/api/accounts/602501000102/transactions | cut -c1-24
{"source":"database"}
$ docker compose up -d api      # restore
```

EXERCISE 2

Prove a HIT never touches PostgreSQL

TASK Use PostgreSQL's own commit counter (pg_stat_database.xact_commit). Show that 20 cache hits leave it unchanged, but 20 misses move it.

Solution

```
$ Q="SELECT xact_commit FROM pg_stat_database WHERE datname='horizonbank'"
$ PSQL="docker exec bank-postgres psql -U bank -d horizonbank -tAc"
$ curl -s localhost:4000/api/accounts/602501000101/transactions >/dev/null # warm
$ $PSQL "$Q"; for i in $(seq 20); do curl -s localhost:4000/api/accounts/602501000101/transactions
>/dev/null; done; $PSQL "$Q"
856
858
# +2 = the two psql calls themselves; the 20 cache hits added nothing
$ $PSQL "$Q"; for i in $(seq 20); do docker exec bank-redis redis-cli DEL bank:txns:602501000101
>/dev/null;
curl -s localhost:4000/api/accounts/602501000101/transactions >/dev/null; done; $PSQL "$Q"
860
901
# +41: two queries (existence check + statement) per miss, plus one psql call
```

EXERCISE 3

Cache the customer profile

TASK GET /api/customers/:id is not cached. Cache it as bank:customer: for 1 hour. The profile never changes in this app, so no invalidation is needed.

Solution

```
// services/accountService.js
export async function getCustomer(customerId) {
  const { data } = await cache.getOrLoad(`bank:customer:${customerId}`, async () => {
    const c = await accounts.findCustomer(customerId);
    if (!c) throw new HttpError(404, `Customer ${customerId} not found`);
    return c;
  });
}
```

```
});
return data;
}
// getOrLoad uses the global TTL; for a 1-hour TTL add an optional ttl argument
// to getOrLoad()/set() and pass 3600 here.
```

EXERCISE 4

Stop a cache stampede

TASK If a hot statement key expires while 200 requests arrive, all 200 run the SQL. Add a lock so only one request loads; the rest wait 50 ms and re-read the cache.

Solution

```
// cache/cacheService.js - inside getOrLoad(), on a miss, before calling loader()
const lockKey = `lock:${key}`;
const gotLock = await redis.set(lockKey, '1', { NX: true, EX: 5 });
if (!gotLock) {
  await new Promise((r) => setTimeout(r, 50));
  const again = await get(key);
  if (again !== null) return { data: again, source: 'cache', ttl: await redis.ttl(key), key };
}
try {
  const data = await loader();
  ... SET key ... (as today)
} finally {
  if (gotLock) await redis.del(lockKey);
}
```

If the lock holder is slow the waiter falls through and loads anyway - the lock reduces the stampede, it does not have to eliminate it. EX 5 guarantees a crashed holder cannot block the key forever.

EXERCISE 5

Explain a design decision

TASK A colleague proposes: "Read the balance from Redis in the transfer service - it is faster." Write the two-sentence answer.

Solution

```
# The cached balance can be up to one TTL old, and two concurrent transfers would both
# read the same cached value - so it can overdraw an account. The balance that authorises
# a debit must come from PostgreSQL under SELECT ... FOR UPDATE; the cache is for display.
```